

Design and Development of a Payment Box based on ESP32 and MQTT

Project Codename: Genesis

Rabijith K R

`rabijithkr002@gmail.com`

Abstract

This paper presents the design, construction, and evaluation of a standalone, voice-announcing IoT payment terminal built around an ESP32 microcontroller, the MQTT publish-subscribe protocol, and the Razorpay payment gateway. Unlike commercial payment sound boxes, whose internal design is proprietary, this work documents a complete, reproducible build: hardware selection and assembly, firmware architecture, cloud-side webhook and QR-generation infrastructure, and a text-to-speech-like mechanism that announces arbitrary transaction amounts from a small library of pre-recorded audio segments. In addition to the build itself, this paper documents several practical engineering challenges encountered during development — an intermittent capacitive touch controller fault, an unreliable microSD storage interface, and a driver-level conflict within the ESP32 Arduino core — along with the diagnostic reasoning used to resolve each, with the intent of helping others building similar systems avoid or more quickly resolve comparable issues. The completed system is evaluated end-to-end using live Razorpay test-mode transactions, with webhook ingestion, MQTT relay, on-device speech synthesis, and persistent transaction logging all confirmed functional.

1 Introduction

Point-of-sale terminals that provide audible confirmation of payment amounts are now common in Indian retail settings, primarily through UPI-linked commercial “sound box” devices distributed by payment aggregators. These devices are closed, proprietary systems; their internal architecture, firmware, and integration details are not publicly documented. This paper describes the design and construction of a functionally similar, fully open, and reproducible system, internally referred to throughout development by the project codename *Genesis*.

The goal of this work is twofold. First, to document a complete and reproducible build — hardware selection, wiring, firmware, and the cloud-side infrastructure needed to connect a physical device to a real payment gateway — so that others attempting a similar project have a working reference rather than starting from an empty specification. Second, to document, honestly and in useful detail, the engineering challenges that arose during development, several of which were not attributable to design errors but to defective components or undocumented software interactions, and which consumed a disproportionate share of development time relative to their eventual, often simple, resolutions.

The remainder of this paper is organized as

follows. Section 2 describes the overall system architecture. Section 3 documents the hardware and firmware build in detail, structured as a practical guide. Section 4 discusses the significant engineering challenges encountered and how each was resolved. Section 5 presents an end-to-end evaluation of the completed system, and Section 6 discusses limitations and future work.

2 System Architecture

The system follows an event-driven, publish-subscribe architecture rather than a polling model, minimizing latency and unnecessary network traffic on the resource-constrained embedded endpoint. Figure 2 illustrates the complete data flow, from customer payment to on-device announcement.

The relay server performs three functions: (i) receipt and cryptographic verification of Razorpay webhook payloads via HMAC-SHA256 signature validation, (ii) on-demand generation of amount-specific QR codes through the Razorpay QR Code API, and (iii) conversion of the resulting raster QR images into the RGB565 pixel format required by the terminal’s display driver, offloading this computation from the microcontroller. The server is deployed on a persis-

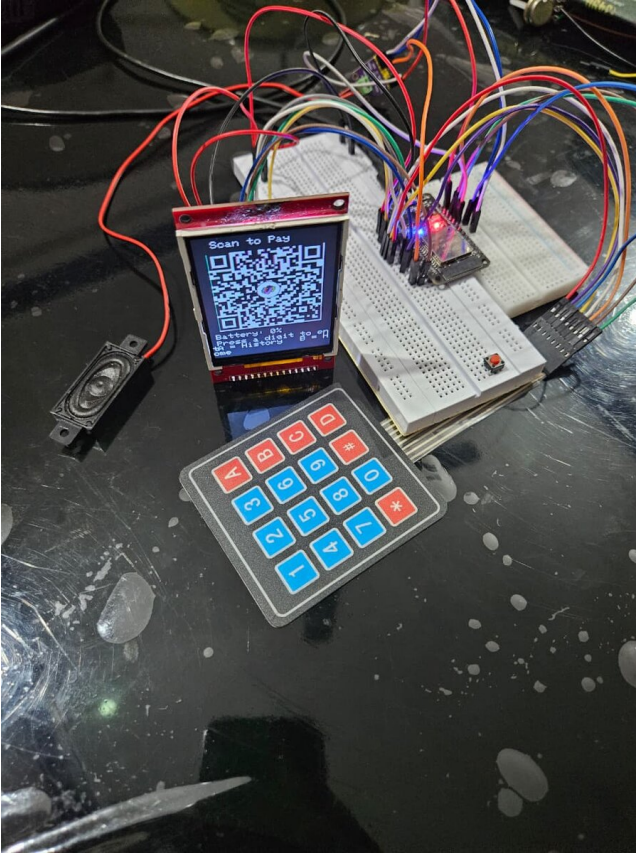


Figure 1: The completed Genesis prototype: display showing the static payment QR code and battery status, matrix keypad for amount entry, and speaker, with the ESP32 and supporting circuitry on the breadboard behind.

tent cloud host (Render) rather than a local development tunnel, with an external monitoring service (UptimeRobot) issuing periodic health-check requests to prevent free-tier suspension due to inactivity.

3 Building the System

3.1 Hardware Components

Table 1 lists the principal hardware subsystems used in the final build. Figure 3 shows a representative subset of the discrete components — the matrix keypad, potentiometers, header pins, and passive components — prior to assembly.

The initial hardware specification included a resistive touchscreen for amount entry and a microSD card for audio asset storage and transaction logging. Both were replaced during development following hardware faults documented in Section 4: touch input was replaced with the matrix keypad shown in Figure 3, and SD-based

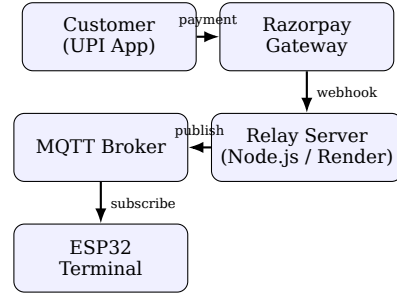


Figure 2: End-to-end event flow from payment initiation to device-side announcement.

Subsystem	Component
Control	ESP32 Dev Module (ESP32-WROOM-32, dual-core, Wi-Fi)
Display	2.8" SPI TFT, ILI9341 driver, 240×320
Audio output	MAX98357A I2S class-D amplifier, 8Ω speaker
User input	4×4 matrix keypad
Power	18650 Li-ion cell, step-up/charge controller

Table 1: Principal hardware subsystems in the final build.

storage was replaced with in-flash asset embedding and non-volatile key-value storage.

3.2 Assembly and Wiring

Development proceeded on a breadboard prototyping platform (Figure 4) to allow iterative wiring changes without soldering, given the number of hardware revisions required (Section 4). The display module communicates with the ESP32 over a shared SPI bus (MOSI, MISO, SCLK, plus dedicated chip-select lines for the display and, in earlier revisions, the SD card and touch controller); the audio amplifier communicates over I2S (BCLK, LRC, DIN); and the matrix keypad and analog potentiometers occupy the remaining available GPIO pins, avoiding ESP32 boot-strapping pins where possible.

Two verification steps proved essential throughout assembly and are recommended as standard practice for similar builds: (i) validating each new peripheral in isolation, using a minimal test sketch, before integrating it with the rest of the firmware; and (ii) using a digital multimeter in continuity mode (Figure 5) to directly verify wiring correctness rather than relying on visual inspection or inference from firmware behavior alone. The latter proved decisive in the touch controller diagnosis described in Section 4.



Figure 3: Discrete components used in the input subsystem: 4×4 matrix keypad, potentiometers, header pins, and passive components (resistors) prior to assembly.

3.2.1 Pin Assignments

Tables 2–4 give the complete ESP32 pin mapping for the display, audio amplifier, and keypad subsystems, to support reproduction of this build.

ESP32 Pin	Display Pin (ILI9341, SPI)
GPIO23	SDI (MOSI)
GPIO19	SDO (MISO)
GPIO18	SCK
GPIO15	CS
GPIO2	D/C
GPIO4	RESET
GPIO32	LED (backlight, PWM)
3.3V	VCC
GND	GND

Table 2: Display module pin assignments.

3.3 Firmware Architecture

The firmware is structured as an event loop coordinating four concerns: network connectivity (Wi-Fi and MQTT), user input (keypad polling), display rendering (screen-state machine covering home, amount-entry, and history views), and audio playback. A key implementation constraint — avoiding the Arduino `analogRead()`

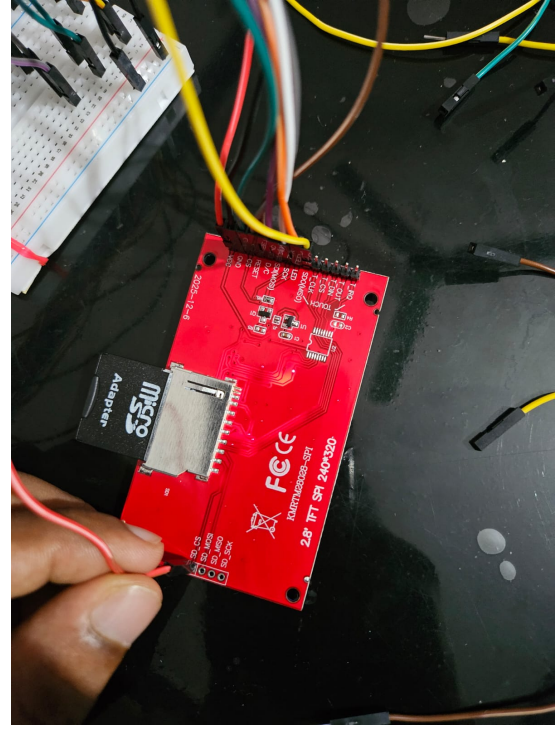


Figure 4: Breadboard-stage wiring during touch controller diagnosis, showing the display module, jumper wiring, and calibration routine executing on-screen.

ESP32 Pin	MAX98357A Amplifier Pin
GPIO26	BCLK
GPIO25	LRC
GPIO22	DIN
5V	Vin
GND	GND

Table 3: I2S audio amplifier pin assignments. Speaker leads connect to the amplifier’s screw terminal.

function in favor of the ESP-IDF legacy ADC driver — is discussed in Section 4, as it is not obvious from documentation alone and caused a significant development delay.

3.4 Speech Synthesis from Segmented Audio

Rather than storing a pre-recorded clip for every possible transaction amount, which does not scale to arbitrary currency values, the firmware constructs spoken amounts by concatenating a library of 34 pre-recorded word segments (digits one through nineteen, multiples of ten from twenty through ninety, and the words *hundred*, *thousand*, *rupee(s)*, and *and*). An amount $A \in [1, 99,999]$ is decomposed as



Figure 5: Digital multimeter used in continuity mode to directly verify signal-line wiring, rather than inferring correctness from firmware behavior.

ESP32 Pin	Keypad Line
GPIO12	Row 1
GPIO13	Row 2
GPIO14	Row 3
GPIO16 (RX2)	Row 4
GPIO17 (TX2)	Column 1
GPIO27	Column 2
GPIO21	Column 3
GPIO5	Column 4

Table 4: 4×4 matrix keypad pin assignments. GPIO16/17 are labeled RX2/TX2 on some ESP32 development boards, corresponding to the unused UART2 peripheral.

$$A = 1000T + 100H + L, \quad L \in [0, 99] \quad (1)$$

where T (thousands) and L (the tens/ones remainder) are each further decomposed using the same two-digit logic, and the corresponding audio segments are played sequentially. This produces natural-sounding announcements (e.g., $347 \rightarrow$ “three hundred and forty seven rupees received”) from a small, fixed asset library.

Following the SD card reliability failure discussed in Section 4, all audio assets (approximately 950 KB of 16-bit, 16 kHz mono PCM data) were converted offline into C header arrays and compiled directly into the firmware image, resident in flash and accessed via `PROGMEM`, eliminating the filesystem layer and its associated failure mode entirely. Figure 6 shows the completed audio output stage.



Figure 6: Assembled audio output stage: MAX98357A I2S amplifier with speaker leads connected via a soldered screw-terminal block.

4 Challenges Encountered During Development

This section documents the three most significant engineering challenges encountered, presented as practical guidance for others building similar systems rather than as a purely academic case study.

4.1 Intermittent Capacitive Touch Failure

The touch controller on the initial display module produced inconsistent calibration values across repeated trials, ranging from near-zero to values exceeding the ADC’s valid range, with no repeatable correlation to physical touch location. Repeated re-wiring and GPIO reassignment did not resolve the issue. Direct continuity measurement of all four touch-interface signal lines (Figure 5), including measurement under physical wire flexion to detect intermittent contact, confirmed the wiring itself was electrically sound. A substitution test on a second, nominally identical display module isolated the fault to the touch controller IC rather than the wiring or firmware. Touch input was subsequently replaced with the matrix keypad shown in Figure 3.

Practical takeaway: when a fault persists despite repeated wiring changes, escalate to

direct electrical measurement before continuing to search the wiring/pin-assignment space; component substitution testing, once wiring is confirmed sound, is a more reliable next step than further parameter search.

4.2 Unreliable microSD Storage

A microSD interface, functional in isolated testing, began failing intermittently once an I2S audio amplifier was added to the same SPI bus. Isolating the amplifier (disconnecting it while leaving SD wiring unchanged) did not resolve the failure, and card/adaptor substitution testing indicated a fault in the SD interface hardware rather than a power or bus-contention issue. Rather than continuing to debug an unreliable removable-storage interface, the design was changed to eliminate the dependency: audio assets were embedded in flash (Section 3), and transaction history was migrated to the ESP32's internal non-volatile key-value store (Preferences/NVS), which requires no external storage medium.

Practical takeaway: for small, fixed-size assets and logs, in-flash storage is a more robust choice than removable media on low-cost development boards, and avoids an entire class of mechanical/electrical connection failures.

4.3 ADC Driver Conflict Producing a Silent Boot Loop

Following integration of potentiometer-based analog input, the device entered a continuous crash-reboot cycle immediately after boot, with no application-level diagnostic output visible under default serial monitor settings. The default auto-scrolling behavior of the development environment's serial console repeatedly hid the actual fault message beneath subsequent boot-log noise; disabling auto-scroll and manually reviewing buffered output revealed:

```
E ADC: CONFLICT! driver_ng is not allowed to be used
with the legacy driver
```

This is a known interaction in recent Arduino-ESP32 core releases: the legacy I2S driver (required for audio) internally depends on the deprecated ADC driver, while the standard `analogRead()` function transparently invokes a newer ADC driver implementation. Concurrent use of both within a single firmware image triggers an unrecoverable abort. The resolution was to replace all analog input calls with explicit legacy-driver calls (`adc1_get_raw()`), matching

the driver generation already required by the audio subsystem.

Practical takeaway: this failure was unrelated to any error in application logic or wiring, and would not have been found without first disabling serial-console auto-scroll. When a fault appears immediately upon integrating a new peripheral subsystem and produces no readable diagnostic output, a driver/HAL-generation mismatch should be considered as a distinct fault class from application-level bugs.

4.4 Additional Practical Issues

Several smaller issues are worth noting briefly, as each is easy to overlook: (i) the ESP32 supports only 2.4 GHz Wi-Fi, and connection failures against a 5 GHz network produce no explicit error indicating the band mismatch; (ii) recent Arduino-ESP32 core releases replaced the channel-based LEDC (PWM) API used for display backlight control with a simpler pin-based API, requiring code updates when upgrading the board package; (iii) stacking a hardware amplifier gain increase with a software amplitude increase produced audible digital clipping, resolved by clamping the software gain stage rather than relying on hardware gain alone; and (iv) the default Arduino partition scheme proved insufficient once approximately 1 MB of embedded audio data was added to the firmware image, requiring selection of a partition scheme allocating additional flash space to the application.

5 System Evaluation

The completed system, shown mid-verification in Figure 7, was evaluated end-to-end using Razorpay's test-mode payment simulation. A simulated payment event was observed to propagate correctly through the full pipeline: web-hook receipt and signature verification at the relay server, MQTT message delivery, and correct on-device speech announcement and persistent history logging. Dynamic, amount-specific QR code generation via the Razorpay API was confirmed functional at the API and rendering level; however, QR codes generated under Razorpay's test-mode environment are, by platform design, not resolvable by production UPI client applications, a limitation confirmed against Razorpay's published documentation and independent of the implementation described here.

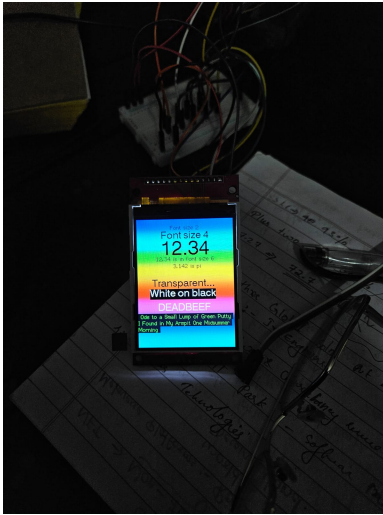


Figure 7: Display module under test, confirming correct SPI communication, color rendering, and multi-size font output prior to full system integration.

6 Limitations and Future Work

A structural limitation of the system, independent of implementation quality, is its dependence on a Razorpay business account for full production operation. While development and testing were conducted successfully under a personal/individual account in test mode, generating and accepting live, UPI-app-scannable QR codes requires either a KYC-verified individual account or a registered business account, along with completed Know Your Customer (KYC) verification. This is a platform-level requirement of the payment gateway rather than a limitation of the system architecture, but it constrains the conditions under which the terminal can be deployed for genuine transaction acceptance, and is a relevant consideration for anyone reproducing this work with the intent of accepting real payments rather than operating purely in a development/test capacity.

Two further aspects of the system remain incomplete at the time of writing. First, analog potentiometer inputs for volume and brightness control exhibited a restricted effective range in initial testing, suggestive of a wiring or component fault not yet root-caused using the methodology of Section 4; a fixed default configuration was substituted as an interim measure. Second, the MQTT transport currently uses a public, unauthenticated broker suitable for development but not recommended for production deployment; migration to a private, authenti-

cated broker is planned prior to production use. Future work includes completing potentiometer fault diagnosis, migrating to production-mode payment gateway credentials once account verification completes, and physical enclosure design.

7 Conclusion

This paper described the design, construction, and evaluation of Genesis, a complete IoT payment announcement terminal integrating embedded firmware, cloud infrastructure, and a third-party payment gateway, and documented three significant hardware and software engineering challenges encountered during development along with their resolutions. The diagnostic approach common to each — preferring direct electrical or log-level measurement over inference, and substituting or redesigning around a component confirmed defective rather than continuing to debug it — is offered as practical guidance for others undertaking similar embedded systems projects.

References

- [1] Razorpay Software Private Limited, “Razorpay API Reference: Webhooks and QR Codes,” <https://razorpay.com/docs/>, accessed 2026.
- [2] OASIS, “MQTT Version 5.0 Specification,” OASIS Standard, 2019.
- [3] Espressif Systems, “ESP32 Technical Reference Manual,” 2024.
- [4] Espressif Systems, “ESP-IDF Programming Guide: Analog-to-Digital Converter (ADC),” <https://docs.espressif.com/>, accessed 2026.
- [5] “Low-Cost ESP32, Raspberry Pi, Node-RED, and MQTT Protocol Based SCADA System,” in *Proc. IEEE Conference*, 2020. <https://ieeexplore.ieee.org/document/9216412/>
- [6] “Evaluating the Energy Consumption of ESP32 Microcontroller for Real-Time MQTT IoT-Based Monitoring System,” in *Proc. IEEE Conference*, 2023. <https://ieeexplore.ieee.org/document/10391358/>
- [7] M. J. A. Baig, M. T. Iqbal, M. Jamil, et al., “Design and implementation of an open-source IoT and blockchain-based peer-to-

peer energy trading platform using ESP32-S2, Node-RED and MQTT protocol," *Energy Reports*, vol. 7, pp. 5733–5746, 2021.

- [8] "Smart Home Automation with MQTT Using ESP32 and Node-RED," in *Proc. 2025 Fourth Int. Conf. on Power, Control and Computing Technologies (ICPC2T)*, 2025.
<https://ieeexplore.ieee.org/document/10958730/>
- [9] National Payments Corporation of India (NPCI), "Unified Payments Interface (UPI) — Product Overview," <https://www.npci.org.in/what-we-do/upi/product-overview>, accessed 2026.